

Exemple

Lorenzo Segoni

1^{er} juillet 2026

Table des matières

1	Bases de Données (BD) et SGBD	1
1.1	Introduction : La problématique des fichiers	1
1.1.1	Limite des systèmes de gestion de fichiers	1
1.2	Définitions Fondamentales	1
1.2.1	Base de Données (BD)	2
1.2.2	Système de Gestion de Base de Données (SGBD)	2
1.3	Objectifs d'une approche Base de Données	2
1.4	Les Fonctions du SGBD	3
1.4.1	Le Langage de Définition des Données (LDD)	3
1.4.2	Le Langage de Manipulation des Données (LMD)	3
1.4.3	Le Contrôle de l'intégrité	3
1.4.4	La Sécurité de fonctionnement	3
2	Le Modèle Conceptuel de Données (E/A)	5
2.1	Le Principe du Modèle E/A	5
2.2	Les Composants Fondamentaux	5
2.2.1	L'Entité (L'objet)	5
2.2.2	Les Attributs (Les détails)	6
2.2.3	L'Identifiant (La clé)	7
2.3	Les Associations (Les liens)	7
2.3.1	Définition	7
2.3.2	Les Cardinalités (La règle du jeu)	8
2.3.3	Comprendre le (min, max)	9
2.3.4	Les 3 grands types de relations	9
2.4	Concepts Complémentaires	9
2.4.1	Entités Faibles	9
2.4.2	Contraintes d'Intégrité (CI)	10
2.5	Formalisation et Documentation	10
2.5.1	La Description des Objets (Entités et Associations)	10
2.5.2	Description d'une Association	10
2.5.3	Le Dictionnaire des Données	10

2.5.4	Les Contraintes d'Intégrité (CI)	11
3	Le langage SQL (SGBD Oracle)	13
3.1	Introduction au SGBD Oracle	13
3.2	Structure générale d'une requête	14
3.2.1	Syntaxe générale de la commande SELECT	14
3.2.2	Oracle-SQL et la casse des caractères	15
3.3	Projection d'une relation	15
3.3.1	La clause DISTINCT	16
3.4	Restriction d'une relation	16
3.4.1	Restriction : syntaxe d'une condition	17
3.4.2	Restriction : exemples	18
3.4.3	Notion de valeur indéfinie (NULL)	18
3.4.4	Restriction : opérateurs IS NULL et LIKE	18
3.4.5	Restriction : opérateur IN	19
3.5	Restriction et projection	19
3.6	Jointure — Produit cartésien	20
3.6.1	Jointure	20
3.6.2	Jointure : exemple de requête	21
3.6.3	Jointure : autre exemple de requête	21
3.6.4	Jointure : utilisation d'alias	21
3.6.5	Quelques règles de nommage	22
3.7	Opérations ensemblistes (UNION , INTERSECT , EXCEPT)	22
3.8	Expression de calcul dans les colonnes projetées	23
3.8.1	Expression de calcul dans la condition (WHERE)	23
3.9	Requêtes imbriquées	24
3.9.1	Requêtes imbriquées : IN / NOT IN	24
3.9.2	Requêtes imbriquées : EXISTS / NOT EXISTS	24
3.10	Fonctions d'agrégation	25
3.11	Partition de relations (GROUP BY)	26
3.11.1	Restriction des groupes : HAVING	27
3.11.2	Partition sur plusieurs colonnes	28
3.11.3	Sous-requête et fonctions d'agrégation	28
3.12	Tri du résultat d'une requête (ORDER BY)	29
3.13	Mise à jour des données	29
3.13.1	Ajout de tuples (INSERT)	29
3.13.2	Modification de tuples (UPDATE)	30
3.13.3	Suppression de tuples (DELETE)	31
3.14	CREATE TABLE	32

3.14.1	Types de données dans ORACLE	33
3.14.2	Contraintes sur le domaine d'un attribut	34
3.14.3	Contrainte de clé primaire (PRIMARY KEY)	35
3.14.4	Création de schéma + instanciation	36

Chapitre 1

Bases de Données (BD) et SGBD

1.1 Introduction : La problématique des fichiers

Avant l'invention des bases de données (années 1960), l'informatique reposait sur des systèmes de gestion de fichiers classiques. Cette méthode présentait des limites majeures qui ont conduit à la création des SGBD

1.1.1 Limite des systèmes de gestion de fichiers

Lorsqu'une application gère ses données via de simples fichiers, nous rencontrons trois problèmes fondamentaux :

- **La Redondance des données** : Les mêmes informations sont souvent répétées dans plusieurs fichiers pour différentes applications.
Conséquence : Gaspillage d'espace et risque d'incohérence (si on modifie une info à un endroit mais pas à l'autre).
- **La Dépendance Programmes / Données** : La structure des données est "codée en dur" dans le programme.
Conséquence : Si l'on change l'organisation physique d'un fichier, il faut réécrire tous les programmes qui l'utilisent. C'est une gestion complexe.
- **La Gestion des accès** : Il est difficile de permettre à plusieurs utilisateurs d'accéder et de modifier le même fichier en même temps sans créer de conflits.

1.2 Définitions Fondamentales

Il est crucial de ne pas confondre le contenu (la base) et le contenant/gestionnaire (le système).

1.2.1 Base de Données (BD)

Définition 1.1. Une Base de Données est une collection de données représentant des informations du monde réel.

Pour être qualifiée de BD, cette collection doit respecter quatre critères :

- **Cohérence et Structure** : Les données suivent un schéma logique défini.
- **Indépendance** : Les données existent indépendamment des applications qui les utilisent.
- **Non-redondance** : On évite de stocker deux fois la même information (redondance minimale).
- **Accessibilité** : Les données sont accessibles par plusieurs utilisateurs simultanément.

1.2.2 Système de Gestion de Base de Données (SGBD)

Définition 1.2. Le SGBD est le logiciel qui sert d'interface entre les utilisateurs (ou applications) et la base de données.

Ses rôles principaux sont :

- La structuration des données.
- Le stockage physique.
- La mise à jour et la consultation.

Exemples de domaines d'application : Gestion d'entreprise, systèmes transactionnels (banques), e-commerce, bibliothèques numériques, etc.

1.3 Objectifs d'une approche Base de Données

L'utilisation d'un SGBD vise à résoudre les problèmes des systèmes de fichiers (vus en partie 1) en atteignant les objectifs suivants :

- **Indépendance Physique et Logique** : Le changement de la structure interne des données ou de leur stockage physique ne doit pas impacter les programmes. C'est l'objectif le plus important.
- **Manipulation aisée** : Permettre à des non-informaticiens d'interroger et de mettre à jour les données facilement.

- **Partage et Sécurité** : Plusieurs applications peuvent utiliser les mêmes données sans conflit.
- **Performance** : Garantir une efficacité d'accès (temps de réponse rapide) même avec de gros volumes de données.

1.4 Les Fonctions du SGBD

Pour atteindre ces objectifs, le SGBD offre quatre fonctions techniques majeures. Il est important de bien distinguer les deux langages (LDD et LMD).

1.4.1 Le Langage de Définition des Données (LDD)

Il permet de décrire la **structure** de la base (le squelette).

- **Rôle** : Définir les objets (tables), leurs attributs (colonnes), les liens entre eux et les contraintes.
- **Résultat** : On obtient le Schéma de la Base de Données.

1.4.2 Le Langage de Manipulation des Données (LMD)

Il permet de gérer le **contenu** de la base (les données elles-mêmes).

- **Rôle** : Créer, modifier, supprimer ou consulter des données.
- **Outil** : C'est ici qu'intervient le langage SQL.

1.4.3 Le Contrôle de l'intégrité

Le SGBD s'assure que les données respectent les règles définies (par le schéma ou le programme). Il empêche l'insertion de données aberrantes.

1.4.4 La Sécurité de fonctionnement

Le SGBD gère les aspects critiques de l'exploitation :

- **Les Transactions et la journalisation** : Assurer que si une opération plante au milieu, on peut revenir en arrière (rollback) pour ne pas corrompre la base.
- **Les Accès concurrents** : Gérer plusieurs utilisateurs en même temps.
- **La Confidentialité** : Gérer les droits d'accès (qui a le droit de voir quoi).

Chapitre 2

Le Modèle Conceptuel de Données (E/A)

Le modèle Entité / Association

2.1 Le Principe du Modèle E/A

Avant de créer des tables dans l'ordinateur, il faut dessiner le schéma sur papier.

- **Origine** : Proposé par Peter Chen en 1976.
- **Objectif** : C'est une représentation graphique standardisée pour décrire les données d'un Système d'Information (SI).
- **Utilité** : Il sert de pont. Une fois le modèle E/A terminé, il est très facile de le traduire en tables SQL.

2.2 Les Composants Fondamentaux

Pour dessiner ce modèle, nous avons besoin de trois briques de base : l'Entité, l'Attribut et l'Identifiant.

2.2.1 L'Entité (L'objet)

Définition 2.1. Une entité est un objet (concret ou abstrait) à propos duquel on souhaite gérer des informations.

Il ne faut pas confondre le "moule" et l'objet créé :

1. **Type d'entité (Le moule)** : C'est la classe générale, le concept.

Exemple : L'entité Étudiant, Client, Département.

2. **Occurrence d'entité (L'individu)** : C'est un élément précis, un individu spécifique qui appartient à ce type.

Exemple : L'employé Alex Térieur ou Paul Auchon.

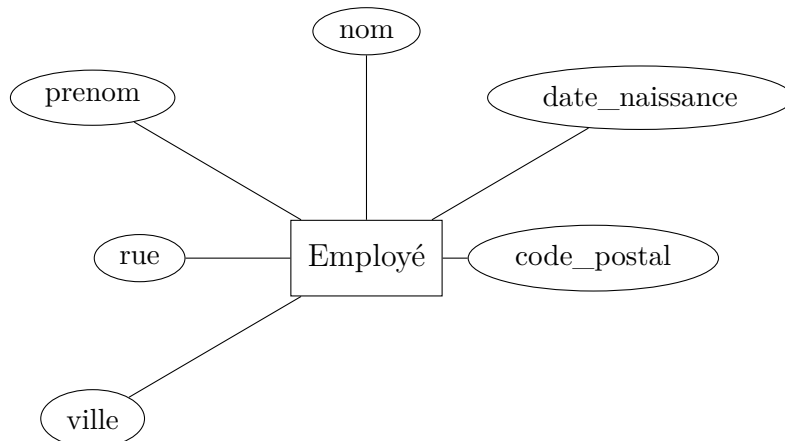
2.2.2 Les Attributs (Les détails)

Définition 2.2. Ce sont les propriétés qui décrivent une entité (ou une association).

Chaque attribut possède :

1. **Un Nom** : (ex : Nom, Prix, Couleur).
2. **Un Domaine** : L'ensemble des valeurs possibles (ex : Entier, Réel positif, Chaîne de caractères, liste de choix Rouge, Vert, Bleu).
3. **Une Occurrence** : La valeur précise pour un individu (ex : "Rouge" est une occurrence de l'attribut Couleur).

Représentation Graphique : Dans le schéma, l'Entité est un Rectangle et les Attributs sont listés à l'intérieur (ou dans des bulles reliées au rectangle).



Que on peut représenter aussi :

Entité : Employé
nom
prénom
date de naissance
rue
code postal
ville

TABLE 2.1 – Tableau Exemple

2.2.3 L'Identifiant (La clé)

C'est le concept le plus important pour retrouver une info précise.

Définition 2.3. L'identifiant est l'ensemble minimal d'attributs qui permet de distinguer de façon unique chaque occurrence.

Évolution de l'identifiant (Exemple du cours) :

1. **Mauvaise pratique :** Utiliser {Nom, Prénom, Date de naissance }. C'est lourd et il y a toujours un risque d'homonyme parfait.

Entité : Employé
<u>nom</u>
<u>prénom</u>
<u>date de naissance</u>
rue
code postal
ville

TABLE 2.2 – Tableau avec la mauvaise pratique

2. **Bonne pratique (Identifiant artificiel) :** On ajoute un attribut dédié, souvent souligné dans le schéma.

Exemple : On ajoute Numero_Employe.

Dans le schéma graphique, l'identifiant est toujours souligné.

Entité : Employé
<u>Numero_Employé</u>
nom
prénom
date de naissance
rue
code postal
ville

TABLE 2.3 – Tableau avec la bonne pratique

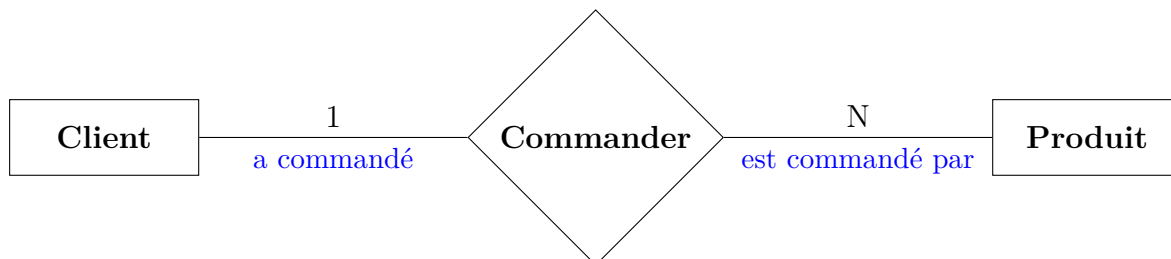
2.3 Les Associations (Les liens)

Les données ne vivent pas seules, elles sont reliées entre elles.

2.3.1 Définition

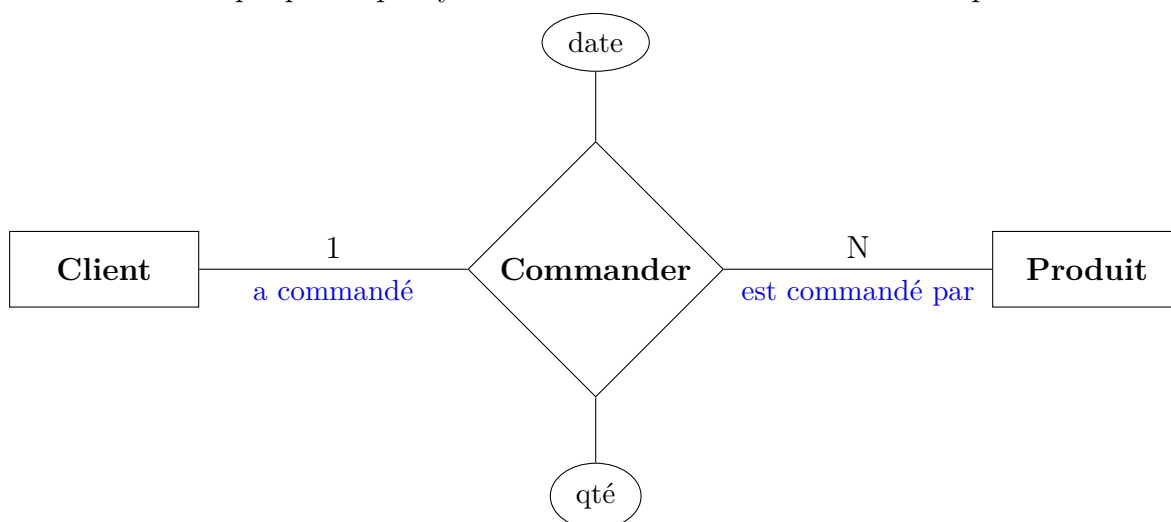
Définition 2.4. Une Association est un lien sémantique entre plusieurs entités. Elle est souvent représentée par un verbe.

— *Exemple :*



— **Attributs d'association :** Parfois, une donnée n'appartient ni à l'un, ni à l'autre, mais au lien lui-même.

Exemple : La **Quantité** (des produits commandé) et la **Date** (de la commande). Elles n'existent que parce qu'il y a une commande entre le client et le produit.

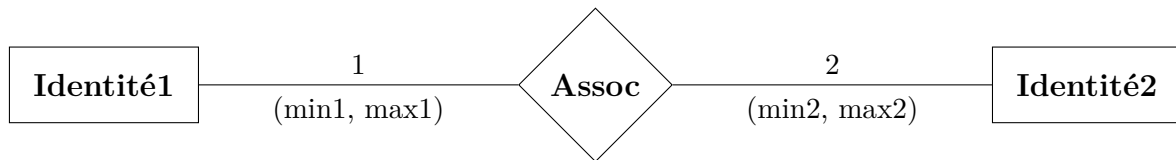


Typologie des Associations

- **Binaire :** Relie 2 entités (le plus courant).
- **Ternaire :** Relie 3 entités.
- **Réflexive :** Une entité est reliée à elle-même (ex : Un employé est marié à un autre employé).

2.3.2 Les Cardinalités (La règle du jeu)

Définition 2.5. Les cardinalités définissent les règles de quantité dans une association. Elles s'écrivent sous la forme (min, max) à côté de chaque entité.



2.3.3 Comprendre le (min, max)

1. **Min (0 ou 1) :** Est-ce que l'entité est obligée de participer ?

- 0 = Non (Optionnel).
- 1 = Oui (Obligatoire).

2. **Max (1 ou N) :** Combien de fois maximum peut-elle participer ?

- 1 = Une seule fois.
- N (ou M) = Plusieurs fois (No limit).

2.3.4 Les 3 grands types de relations

On classe les associations selon leur cardinalité **maximale** (le chiffre de droite) des deux côtés :

1. **Relation 1-1 (One-to-One) :**

Une occurrence de E est liée à **une seule** occurrence de F, et inversement.

2. **Relation 1-N (One-to-Many) :**

D'un côté, c'est unique (1), de l'autre c'est multiple (N).

Exemple : Un **Auteur** écrit plusieurs **Livres**, mais un **Livre** est écrit par un seul **Auteur** (dans ce modèle simplifié).

3. **Relation N-M (Many-to-Many) :**

Plusieurs des deux côtés.

Exemple : Un **Client** commande plusieurs **Produits**, et un **Produit** peut être commandé par plusieurs **Clients**.

2.4 Concepts Complémentaires

2.4.1 Entités Faibles

Définition 2.6. Une entité qui ne possède pas d'identifiant propre.

- Elle ne peut exister que si elle est rattachée à une "Entité Forte".
- Son identifiant est composé de celui de l'entité forte + un identifiant partiel.

La cardinalité vers l'entité forte est toujours **(1,1)** (dépendance totale).

2.4.2 Contraintes d'Intégrité (CI)

Ce sont des règles pour garantir que les données restent logiques.

- **CI Statiques** : Doivent être vraies tout le temps (ex : Le **Nom** est obligatoire, **Date naissance** < **Date mariage** , **Fax** attributs facultatifs, ...).
- **CI Dynamiques** : Règles logiques sur des valeurs (ex : le **salaire** ne peut qu'augmenter).

2.5 Formalisation et Documentation

Pour qu'un projet de base de données soit valide, il ne suffit pas de faire un dessin (le schéma). Il faut respecter des **règles de complétude** : chaque objet dessiné doit être décrit textuellement de manière exhaustive.

L'ensemble de ces descriptions constitue l'**Univers du Discours** (le résumé synthétique de l'application).

2.5.1 La Description des Objets (Entités et Associations)

Chaque élément graphique doit avoir sa fiche d'identité textuelle.

Nom	Auteur
Définition (Contexte)	Personne ayant écrit un livre référencé par l'éditeur
Liste d'attributs	{nom, prénom, adresse}
Identifiant	{nom, prénom}

TABLE 2.4 – Description d'une Entité

2.5.2 Description d'une Association

Prenons l'association **Écriture** qui relie les auteurs aux livres.

2.5.3 Le Dictionnaire des Données

Le dictionnaire des données descend au niveau le plus fin : l'**attribut**. Il précise le format et les règles de chaque donnée.

Nom	Ecriture
Définition	L'écriture associe les livres à l'auteur qui les a écrits
Entités	Auteur, Livre
Rôles & Cardinalités	Un Auteur écrit 1 à N Livres. Un Livre est écrit par 1 à 1 Auteur (1-1)
Attributs propres	$\emptyset$ (Aucun attribut sur l'association elle-même)

TABLE 2.5 – Description d'une Association

Nom	Ville auteur
Définition	Nom de la ville où réside un auteur.
Structure	Atomique (Mono-valué)
Rôles / cardinalité	Chaîne de caractères alphabétiques.
Obligatoire ?	Non. On peut créer un auteur sans connaître sa ville.

TABLE 2.6 – Description d'un Attribut

2.5.4 Les Contraintes d'Intégrité (CI)

Certaines règles logiques ne peuvent pas être dessinées sur le schéma. Il faut les écrire sous forme d'expressions logiques ou mathématiques.

Nom de la contrainte	Existence d'un mariage
Éléments concernés	Association : Mariage , Attribut : 'Âge de l'entité
Expression logique (La règle)	Une occurrence de l'association Mariage n'est valide que si : $Age($

TABLE 2.7 – Contrainte sur une Association

Chapitre 3

Le langage SQL (SGBD Oracle)

Le langage **SQL** (Structured Query Language) est le langage standard utilisé pour interagir avec une base de données relationnelle. Il se compose de trois parties :

- **LDD** (Langage de Définition des Données) : permet de créer, modifier ou supprimer la structure des tables (schémas).
Exemples : `CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`.
- **LMD** (Langage de Manipulation des Données) : permet d'ajouter, supprimer, modifier et interroger les données.
Exemples : `INSERT`, `DELETE`, `UPDATE`, `SELECT`, `COMMIT/ROLLBACK`.
- **LCD** (Langage de Contrôle des Données) : utilisé pour gérer les droits d'accès.
Exemples : `GRANT`, `REVOKE`.

3.1 Introduction au SGBD Oracle

Oracle est un **système de gestion de bases de données relationnelles** (SGBDR), développé par Oracle Corporation depuis 1981.

La version de référence du cours est **Oracle 12c** (sortie en 2013), suivie par Oracle 18c et 19c.

Oracle constitue un **écosystème logiciel complet**, comprenant notamment :

- des serveurs de données et utilitaires (SQL*Plus, import/export loader, ...);
- des environnements de développement (web, Java, pré-compilateurs, ...);
- des outils d'aide à la décision (data warehouse, Oracle Discoverer, ...);
- des progiciels client-serveur et Internet (Oracle Applications, Oracle e-Business Suite).
- ...

3.2 Structure générale d'une requête

Pour illustrer les requêtes SQL, on utilise une base de données exemple composée des tables suivantes :

- **Client** (noClient, nom, prénom, ddn, rue, CP, ville)
- **Produit** (noProduit, libellé, prixUnitaire, *noFournisseur*)
- **Fournisseur** (noFournisseur, raisonSociale)
- **Commande** (*noClient*, *noProduit*, dateCommande , quantité)

Les attributs soulignés désignent les **clés primaires** ; les attributs en *italique* correspondent aux **clés étrangères**.

Conventions de notation

- [élément] : élément optionnel.
- élément1 | élément2 : choisir l'un des deux éléments.
- { élémentx | élémenty } : liste de choix possibles.
- {élémentx} | {élémenty} : choix entre deux ensembles.

3.2.1 Syntaxe générale de la commande SELECT

La commande SELECT permet d'interroger les données d'une ou plusieurs tables. Sa syntaxe générale est :

SQL

```
1 SELECT * | {[ALL | DISTINCT]
2     expression [[AS] nomColonne]
3     [, expression [[AS] nomColonne]]... }
4 FROM relation [alias] [, relation [alias] ...]
5 [WHERE condition]
6 [GROUP BY nomColonne [, nomColonne]...]
7 [HAVING condition]
8 [ORDER BY nomColonne [ASC | DESC]
9     [, nomColonne [ASC | DESC] ...]];
```

Deux commandes SELECT peuvent être combinées :

SQL

```
1 CdeSelect {UNION | INTERSECT | EXCEPT} CdeSelect
```

3.2.2 Oracle-SQL et la casse des caractères

Dans Oracle, il n'existe **aucune différence** entre identifiants écrits en majuscules ou en minuscules : les noms de tables, attributs ou contraintes sont insensibles à la casse.

La seule exception concerne la **comparaison de chaînes de caractères**, où 'Dupont' et 'dupont' sont considérés comme différents.

Exemple :

SQL

```
1 SELECT *
2 FROM Client
3 WHERE nom = 'Dupont';
```

Les instructions suivantes sont équivalentes :

SQL

```
1 SELECT *
2 FROM CLIENT
3 WHERE NOM = 'Dupont';
```

3.3 Projection d'une relation

La **projection** consiste à extraire seulement certains attributs d'une relation. Par exemple, pour récupérer les **numéros de clients** et les **dates de commande** de toutes les commandes, on utilise :

SQL

```
1 SELECT noClient, dateCommande
2 FROM Commande;
```

Résultat :

noClient	dateCommande
100	04/05/2003
200	12/04/2003
100	05/01/2004
300	25/02/2004
400	30/01/2004
400	30/01/2004

Cette requête est équivalente à :

SQL

```
1 SELECT ALL noClient, dateCommande
2 FROM Commande;
```

3.3.1 La clause DISTINCT

Pour éliminer les doublons, on utilise la clause **DISTINCT**. La requête suivante retourne chaque couple (noClient, dateCommande) une seule fois :

SQL

```
1 SELECT DISTINCT noClient, dateCommande
2 FROM Commande;
```

Résultat :

noClient	dateCommande
100	04/05/2003
200	12/04/2003
100	05/01/2004
300	25/02/2004
400	30/01/2004

Expression algébrique équivalente :

$$\pi_{\text{noClient, dateCommande}}(\text{Commande})$$

3.4 Restriction d'une relation

La **restriction** permet de sélectionner les tuples satisfaisant une condition logique. Exemple : on veut les articles dont le prix est inférieur à 20 € et dont le numéro est supérieur à 30 :

SQL

```

1 SELECT *
2 FROM Article
3 WHERE prixUnitaire < 20 AND noArticle > 30;

```

Résultat :

noArticle	libellé	prixUnitaire
31	Brosse	12
40	Tableau	9.99
50	Visseuse	19.99

Expression algébrique équivalente :

$$\sigma_{\text{prixUnitaire} < 20 \wedge \text{noArticle} > 30}(\text{Article})$$

3.4.1 Restriction : syntaxe d'une condition

La syntaxe générale est :

SQL

```

1 SELECT *
2 FROM nomRelation
3 WHERE condition;

```

Les formes possibles d'une condition :

- conditionSimple
- (condition)
- NOT (condition)
- condition AND condition
- condition OR condition

Une conditionSimple peut être de la forme :

- expression =, <, >, <=, >=, <> expression
- expression [NOT] BETWEEN valeur1 AND valeur2
- expression IS [NOT] NULL
- expression [NOT] IN (liste)
- expression [NOT] LIKE patron

3.4.2 Restriction : exemples

Produits dont le prix est compris entre 50 et 100 €

SQL

```
1 SELECT *
2 FROM Produit
3 WHERE prixUnitaire >= 50
4     AND prixUnitaire <= 100;
```

Produits dont le prix est inférieur à 50 € ou supérieur à 100 €

SQL

```
1 SELECT *
2 FROM Produit
3 WHERE prixUnitaire < 50
4     OR prixUnitaire > 100;
```

Écriture équivalente avec BETWEEN :

SQL

```
1 SELECT *
2 FROM Produit
3 WHERE prixUnitaire BETWEEN 50 AND 100;
```

3.4.3 Notion de valeur indéfinie (NULL)

Une valeur NULL indique qu'un attribut est **inconnu** ou **indéfini**. Règles importantes :

- On ne peut tester NULL qu'avec : IS NULL ou IS NOT NULL.
- Toute expression arithmétique impliquant NULL retourne NULL.
- Toute comparaison impliquant NULL retourne UNKNOWN.

3.4.4 Restriction : opérateurs IS NULL et LIKE

Commandes dont la quantité est indéterminée

SQL

```
1 SELECT *
2 FROM Commande
3 WHERE quantite IS NULL;
```

Clients dont le nom commence par B, finit par B et contient au moins 3 caractères

SQL

```
1 SELECT *
2 FROM Client
3 WHERE nom LIKE 'B_\\%B';
```

Rappels sur les patrons LIKE :

- % représente une suite de 0 à n caractères.
- _ représente exactement un caractère.

3.4.5 Restriction : opérateur IN

Clients dont le nom appartient à un ensemble donné

SQL

```
1 SELECT prenom
2 FROM Client
3 WHERE nom IN ('Dupond', 'Durant', 'Martin');
```

Clients dont le nom n'appartient pas à cet ensemble

SQL

```
1 SELECT prenom
2 FROM Client
3 WHERE nom NOT IN ('Dupond', 'Durant', 'Martin');
```

3.5 Restriction et projection

On peut combiner restriction et projection. Exemple : obtenir les numéros de clients et dates de commande pour les commandes postérieures au 01/01/2004 :

SQL

```

1 SELECT noClient, dateCommande
2 FROM Commande
3 WHERE dateCommande > '01/01/2004';

```

Résultat :

noClient	dateCommande
100	05/01/2004
300	25/02/2004
400	30/01/2004

Expression algébrique équivalente :

$$\pi_{\text{noClient}, \text{dateCommande}}(\sigma_{\text{dateCommande} > '01/01/2004'}(\text{Commande}))$$

3.6 Jointure — Produit cartésien

La jointure débute par l'opération la plus simple : le **produit cartésien**. Lorsqu'on liste plusieurs relations dans la clause **FROM**, sans condition supplémentaire, SQL génère toutes les combinaisons possibles de leurs tuples.

— Exemple : produire toutes les paires possibles $\text{Client} \times \text{Commande}$.

SQL

```

1 SELECT *
2 FROM Client, Commande;

```

Expression algébrique équivalente :

$$\text{Client} \times \text{Commande}$$

3.6.1 Jointure

Une jointure SQL correspond en fait à la combinaison :

$$\text{Produit cartésien} + \text{Restriction} + \text{Projection}$$

Forme générale :

SQL

```
1 SELECT attribut1 [, attribut2, ... ]  
2 FROM relation1, relation2 [, relation3, ...]  
3 WHERE condition;
```

Expression algébrique équivalente :

$$\pi_{\text{attributs}}(\sigma_{\text{condition}}(\text{relation1} \times \text{relation2} \times \dots))$$

3.6.2 Jointure : exemple de requête

Afficher la liste des commandes accompagnées du nom du client :

SQL

```
1 SELECT nom, Client.noClient,  
2         noProduit, dateCommande, quantite  
3 FROM Commande, Client  
4 WHERE Client.noClient = Commande.noClient;
```

Remarque : lorsqu'un attribut existe dans plusieurs relations, il faut le préfixer par le nom de la relation pour éviter toute ambiguïté.

3.6.3 Jointure : autre exemple de requête

Produire la liste des produits commandés en quantité supérieure à 100 et dont le prix dépasse 1000 €. On affiche : numéro du produit, libellé, prix unitaire, date de commande.

SQL

```
1 SELECT Produit.noProduit, libelle, prixUnitaire, date  
2 FROM Produit, Commande  
3 WHERE quantite > 100  
4        AND prixUnitaire > 1000  
5        AND Produit.noProduit = Commande.noProd;
```

3.6.4 Jointure : utilisation d'alias

Les alias permettent d'alléger l'écriture, surtout lorsqu'on joint plusieurs relations.
Exemple : afficher les commandes avec le nom et le numéro du client.

SQL

```
1 SELECT C2.noClient, nom, date, quantite
2 FROM Commande C1, Client C2
3 WHERE C1.noClient = C2.noClient;
```

— Commande est renommée C1

— Client est renommée C2

3.6.5 Quelques règles de nommage

Nom d'une colonne dans le résultat :

— Par défaut : nom de l'attribut correspondant.

SQL

```
1 SELECT * FROM Produit;
```

Résultat : noProduit, libellé, prixUnitaire

Renommage possible :

SQL

```
1 SELECT noProduit AS "Numero_ produit"
2 FROM Produit;
```

Résultat : colonne Numéro produit.

3.7 Opérations ensemblistes (UNION, INTERSECT, EXCEPT)

On peut combiner deux commandes SELECT avec les opérateurs :

— UNION : union ensembliste

— INTERSECT : intersection

— EXCEPT : différence

Exemple : trouver les employés qui sont aussi passagers.

Employé		Passager	
10	Henry John	4	Harry Peter
15	Conrad James	78	Conrad James
35	Jenqua Jessica	9	Land Robert
46	Leconte Jean	466	Leconte Jean

Requête :

SQL

```
1 (SELECT nomEmp AS nom, prenomEmp AS prenom
2   FROM Employe)
3 INTERSECT
4 (SELECT nomPass AS nom, prenomPass AS prenom
5   FROM Passager);
```

3.8 Expression de calcul dans les colonnes projetées

On peut intégrer des expressions arithmétiques directement dans la liste des résultats.

Exemple : afficher le prix TTC (taxe 15%) :

SQL

```
1 SELECT noArticle, prixUnitaire,
2       prixUnitaire * 1.15 AS prixTTC
3 FROM Article;
```

3.8.1 Expression de calcul dans la condition (WHERE)

Les expressions peuvent aussi apparaître dans les conditions.

Produits dont le prix TTC dépasse 20 € :

SQL

```
1 SELECT noArticle
2 FROM Article
3 WHERE prixUnitaire * 1.15 > 20;
```

Elles peuvent également utiliser des fonctions prédéfinies.

Exemple : commandes du jour :

SQL

```
1 SELECT *
2 FROM Commande
3 WHERE dateCommande = CURRENT_DATE;
```

3.9 Requêtes imbriquées

Une requête peut apparaître à l'intérieur d'une autre, notamment dans la clause **WHERE**. On parle alors de **sous-requête** ou **requête imbriquée**.

Forme générale :

SQL

```
1 SELECT colonne(s)
2 FROM relation(s)
3 WHERE expression [NOT] IN (sous-requete)
4     OR EXISTS (sous-requete)
5     OR NOT EXISTS (sous-requete);
```

3.9.1 Requêtes imbriquées : IN / NOT IN

Exemple : noms des clients ayant passé une commande le 24/10/2000.

SQL

```
1 SELECT nom
2 FROM Client
3 WHERE noClient IN (SELECT noClient
4                     FROM Commande
5                     WHERE dateCommande = '24/10/2000');
```

3.9.2 Requêtes imbriquées : EXISTS / NOT EXISTS

Clients ayant passé au moins une commande :

SQL

```
1 SELECT *
2 FROM Client C1
3 WHERE EXISTS (
4     SELECT *
5     FROM Commande C2
6     WHERE C1.noClient = C2.noClient
7 );
```

Clients n'ayant passé aucune commande :

SQL

```
1 SELECT *
2 FROM Client C1
3 WHERE NOT EXISTS (
4     SELECT *
5     FROM Commande C2
6     WHERE C1.noClient = C2.noClient
7 );
```

3.10 Fonctions d'agrégation

Les fonctions d'agrégation permettent d'appliquer un calcul sur un ensemble de valeurs (souvent un groupe de lignes) pour produire une valeur unique.

La forme générale est :

SQL

```
1 SELECT fctAgregation
2 FROM relation(s)
3 [WHERE condition];
```

Les principales fonctions sont :

- **COUNT(*)** : nombre total de lignes du résultat.
- **COUNT([DISTINCT] expr)** : nombre de valeurs non NULL (distinctes éventuellement).
- **SUM(n)** : somme des valeurs de n (ignore les NULL).
- **MAX(n)** : valeur maximale.

- **MIN(*n*)** : valeur minimale.
- **AVG(*n*)** : moyenne des valeurs (ignore les NULL).

Ici, *n* désigne une expression numérique et *expr* toute expression valide.

Exemples

Nombre total d'articles et prix moyen :

SQL

```
1 SELECT COUNT(*) AS nbArticles ,
2       AVG(prixUnitaire) AS prixMoyen
3 FROM Article;
```

Nombre de prix non NULL :

SQL

```
1 SELECT COUNT(prixUnitaire) AS nbPrixNonNull
2 FROM Article;
```

Nombre de prix distincts :

SQL

```
1 SELECT COUNT(DISTINCT prixUnitaire) AS nbPrix
2 FROM Article;
```

3.11 Partition de relations (GROUP BY)

La clause **GROUP BY** permet de regrouper les lignes selon une ou plusieurs colonnes, puis d'appliquer une fonction d'agrégation sur chaque groupe.

Exemple

Nombre de produits commandés par client :

SQL

```
1 SELECT noClient, COUNT(*) AS totalProduits
2 FROM Commande
3 GROUP BY noClient;
```

Principe :

1. Les commandes sont regroupées selon `noClient`.
2. Pour chaque groupe, on calcule le nombre d'éléments.

Important : *chaque expression présente dans le SELECT doit avoir une valeur unique par groupe.*

Exemple avec condition préalable

Quantité totale commandée par client, hors produit F565 :

SQL

```
1 SELECT noClient, SUM(quantite)
2 FROM Commande
3 WHERE noProduit <> 'F565'
4 GROUP BY noClient;
```

Étapes :

1. La clause WHERE élimine les commandes du produit F565.
2. Les lignes restantes sont regroupées par client.
3. Pour chaque groupe, la somme des quantités est calculée.

3.11.1 Restriction des groupes : HAVING

La clause HAVING agit comme un filtre, mais **sur les groupes**. Elle ne s'utilise qu'en présence d'un GROUP BY.

Exemple

Quantité moyenne commandée par produit pour les produits ayant reçu plus de 3 commandes (en ignorant le client C47) :

SQL

```
1 SELECT noProduit, AVG(quantite)
2 FROM Commande
3 WHERE noClient <> 'C47'
4 GROUP BY noProduit
5 HAVING COUNT(*) > 3;
```

Étapes :

1. Élimination des commandes du client C47 (WHERE).
2. Regroupement par produit.
3. Élimination des groupes comptant moins de 3 commandes (HAVING).
4. Calcul de la moyenne pour les groupes restants.

3.11.2 Partition sur plusieurs colonnes

Il est possible de grouper sur plusieurs attributs.

Exemple

Nombre de produits commandés par client *et par date* :

SQL

```
1 SELECT noClient, dateCommande, COUNT(noProduit) AS nbProduits
2 FROM Commande
3 GROUP BY noClient, dateCommande;
```

Chaque combinaison (noClient, dateCommande) forme un groupe distinct.

3.11.3 Sous-requête et fonctions d'agrégation

Une fonction d'agrégation ne peut pas être utilisée directement dans un SELECT si elle produit un résultat incompatible avec les autres colonnes.

Exemple incorrect :

SQL

```
1 SELECT noProduit, MAX(prixUnitaire)
2 FROM Produit;
```

Cette requête est invalide car `noProduit` n'est pas unique.

Pour contourner, on utilise une sous-requête :

SQL

```
1 SELECT noProduit, libelle
2 FROM Produit
3 WHERE prixUnitaire > (
4     SELECT AVG(prixUnitaire)
5     FROM Produit
6 );
```

3.12 Tri du résultat d'une requête (ORDER BY)

La clause `ORDER BY` permet de trier le résultat d'une requête selon une ou plusieurs colonnes.

- **ASC** : ordre croissant (défaut)
- **DESC** : ordre décroissant

Exemple

Liste des commandes triées par numéro de client (croissant), puis par date (décroissant) :

SQL

```
1 SELECT *
2 FROM Commande
3 ORDER BY noClient, dateCommande DESC;
```

3.13 Mise à jour des données

Les opérations de mise à jour permettent d'ajouter, de modifier ou de supprimer des tuples dans une relation. Elles s'effectuent à l'aide des instructions `INSERT`, `UPDATE` et `DELETE`.

3.13.1 Ajout de tuples (INSERT)

L'instruction `INSERT` permet d'insérer un ou plusieurs tuples dans une relation. Sa forme générale est la suivante :

SQL

```
1 INSERT INTO nom_relation [(attribut, attribut, ...)]  
2 VALUES (valeur, valeur, ...)
```

ou bien :

SQL

```
1 INSERT INTO nom_relation [(attribut, attribut, ...)]  
2 commande-SELECT
```

Exemple : ajout d'un produit

SQL

```
1 INSERT INTO Produit VALUES (430, 'lecteur_DVD', 9.99);
```

Tous les attributs doivent être fournis dans le même ordre que dans la définition de la table.

Exemple : insertion provenant d'un SELECT

SQL

```
1 INSERT INTO Commande (noClient, noProduit)  
2 SELECT noClient, noProduit FROM Produit, Client;
```

Les attributs non mentionnés dans la liste sont automatiquement positionnés à NULL.

3.13.2 Modification de tuples (UPDATE)

L'instruction UPDATE permet de modifier la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition.

SQL

```
1 UPDATE nom_relation  
2 SET attribut = expression  
3   [, attribut = expression ...]  
4 [WHERE condition];
```

Exemple : modifier le nom du client n°3

SQL

```
1 UPDATE Client
2 SET nom = 'Durand'
3 WHERE noClient = 3;
```

Exemple : augmenter de 5% le prix de certains produits

SQL

```
1 UPDATE Produit
2 SET prixUnitaire = prixUnitaire * 1.05
3 WHERE libelle IN ('CD-ROM', 'DVD', 'ZIP');
```

Exemple : mise à jour basée sur une sous-requête Augmenter de 10% les prix des produits fournis par le fournisseur « Titu » :

SQL

```
1 UPDATE Produit p
2 SET prixUnitaire = prixUnitaire * 1.1
3 WHERE p.noFournisseur IN (
4     SELECT f.noFournisseur
5     FROM Fournisseur f
6     WHERE f.raisonSociale = 'Titu'
7 );
```

Autre exemple Positionner le libellé du produit n°99 et lui attribuer le prix moyen :

SQL

```
1 UPDATE Produit
2 SET libell " " = "produitTest",
3     prixUnitaire = (SELECT AVG(prixUnitaire) FROM Produit)
4 WHERE noProduit = 99;
```

3.13.3 Suppression de tuples (DELETE)

L'instruction DELETE supprime les tuples correspondant à une condition.

SQL

```
1 DELETE FROM relation
2 [WHERE condition];
```

Exemple : supprimer les clients habitant Metz

SQL

```
1 DELETE FROM Client
2 WHERE ville = 'Metz';
```

Exemple : suppression totale

SQL

```
1 DELETE FROM Client;
```

Toutes les lignes sont supprimées, mais le schéma de la table demeure.

3.14 CREATE TABLE

L'instruction `CREATE TABLE` permet de créer une nouvelle table en précisant ses attributs et ses contraintes.

SQL

```
1 CREATE TABLE nomTable (
2     definitionAttribut,
3     ...
4     definitionContrainte
5 );
```

Syntaxe de définition d'un attribut

SQL

```

1 nomAttribut type
2     [DEFAULT valeur]
3     [NULL | NOT NULL]
4     [UNIQUE | PRIMARY KEY]
5     [REFERENCES autreTable(attribut)]
6     [CONSTRAINT nom CHECK (condition)]

```

Il n'y a pas de distinction entre minuscules et majuscules dans les noms.

Syntaxe de définition d'une contrainte

SQL

```

1 [CONSTRAINT nom]
2 PRIMARY KEY (liste_attributs)
3 UNIQUE (liste_attributs)
4 FOREIGN KEY (liste_attributs)
5     REFERENCES tableBis (liste_attributs)
6     [ON DELETE regle]
7     [ON UPDATE regle]
8 CHECK (condition)

```

3.14.1 Types de données dans ORACLE

Types numériques Les types ANSI INT, SMALLINT, FLOAT, REAL... sont convertis en type ORACLE NUMBER.

Type SQL ANSI	Type ORACLE
NUMERIC(p,c), DECIMAL(p,c)	NUMBER(p,c)
INTEGER, SMALLINT	NUMBER(38)
FLOAT(b), REAL	NUMBER

Chaînes de caractères

- CHAR(n) : taille fixe ($n \leq 2000$)
- VARCHAR(n) : taille variable ($n \leq 4000$)

3.14.2 Contraintes sur le domaine d'un attribut

Contrainte NOT NULL L'utilisateur doit fournir une valeur.

SQL

```
1 CREATE TABLE Client (  
2     noClient INTEGER NOT NULL,  
3     nom VARCHAR(50) NOT NULL,  
4     ddn VARCHAR(15),  
5     telephone VARCHAR(15)  
6 );
```

Contrainte CHECK sur un attribut

SQL

```
1 CREATE TABLE Client (  
2     noClient INTEGER NOT NULL  
3         CHECK (noClient > 0 AND noClient < 1000),  
4     nom VARCHAR(50) NOT NULL,  
5     ddn VARCHAR(15)  
6 );
```

Contrainte CHECK sur plusieurs attributs

SQL

```
1 CREATE TABLE Produit (  
2     noProduit INTEGER NOT NULL,  
3     libelle VARCHAR(50),  
4     prixUnitaire NUMBER(10,2) NOT NULL,  
5     CHECK (noProduit <= 100 OR prixUnitaire > 15)  
6 );
```

Valeur par défaut

SQL

```
1 CREATE TABLE employe (  
2     numEmp INTEGER,  
3     numTel VARCHAR(15) DEFAULT 'confidentiel' NOT NULL  
4 );
```


3.14.3 Contrainte de clé primaire (PRIMARY KEY)

La clé primaire identifie de manière unique chaque tuple et ne peut contenir de valeur NULL.

SQL

```
1 CREATE TABLE Client (  
2     noClient INTEGER PRIMARY KEY,  
3     nom VARCHAR(50) NOT NULL,  
4     ddn VARCHAR(15)  
5 );
```

Clé primaire composée

SQL

```
1 CREATE TABLE Commande (  
2     noClient INTEGER,  
3     noProduit INTEGER,  
4     dateCommande DATE,  
5     quantite INTEGER,  
6     PRIMARY KEY (noClient, noProduit, dateCommande)  
7 );
```

Contrainte d'unicité (UNIQUE)

SQL

```
1 CREATE TABLE Citoyen (  
2     numSecu INTEGER PRIMARY KEY,  
3     nom VARCHAR(50),  
4     prenom VARCHAR(50),  
5     ddn DATE,  
6     noPassport INTEGER UNIQUE  
7 );
```

Contrainte d'intégrité référentielle (FOREIGN KEY)

SQL

```
1 CREATE TABLE Commande (  
2     noCommande INTEGER PRIMARY KEY,  
3     noClient INTEGER REFERENCES Client(noClient),  
4     noProduit INTEGER,  
5     dateCommande DATE,  
6     quantite INTEGER,  
7     FOREIGN KEY (noProduit)  
8         REFERENCES Produit(noProduit)  
9 );
```

Les tables référencées doivent exister et posséder une clé primaire ou une clé unique.

3.14.4 Création de schéma + instanciation

SQL

```
1 CREATE TABLE nomTable [(attributs)]  
2 AS SELECT ...
```

Cette syntaxe :

1. crée la structure de la table,
2. insère les résultats d'un SELECT.

Exemple :

SQL

```
1 CREATE TABLE Client54 AS  
2 SELECT noClient, nom  
3 FROM Client  
4 WHERE CP BETWEEN 54000 AND 54999;
```